NET-NEUTRAL AI

App Flow Document

Complete system interaction flows — setup through shutdown

GitHub DevDays Hackathon 2026 | Version 1.0 | May 2026

Companion Docs	PRD v1.0 + TRD v1.0
Flows Covered	Setup, Pre-training, Registration, Training Rounds, Error Handling, Shutdown
Actors	Coordinator (Machine A), Client A, Client B, Client C, Operator
Total Rounds	5 per demo session
Trigger Model	Manual — Operator types command to start each round

1. Actors & Responsibilities

Actor	Machine	Role in Flow
Operator	Any machine (typically Machine A)	Types commands to start server, trigger rounds, initiate shutdown
Coordinator	Machine A (ENTC1 GPU laptop)	Runs Flask server — manages rounds, FedAvg, credits, evaluation
Client A	Machine A (same machine as coordinator)	Trains locally, submits weights, polls for round signals
Client B	Machine B (Janhvesh — CS laptop)	Trains locally, submits weights, polls for round signals
Client C	Machine C (3rd member laptop)	Trains locally, submits weights, polls for round signals

Reading the Flow Tables

Each flow table has 4 columns: step number, actor performing the action, what they do, and what the system responds with. Green cells are system responses. Blue step numbers are the happy path. Error flows are covered separately in Section 6.

2. Flow 0 — Pre-Demo Setup (Done Before Recording Starts)

This flow happens once before the demo video is recorded. It is not shown on camera. Its output — the pre-trained model checkpoint — is the starting point for the demo.

2.1 Environment Setup — Each Machine

#	Actor	Action	System Response
1	Operator	Clone GitHub repo on all machines: git clone [repo-url]	Repo cloned to local directory on all 4 machines
2	Operator	Activate virtual environment: venv\Scripts\activate	Python environment active with correct packages
3	Operator	Install dependencies: pip install -r requirements.txt	All packages installed — torch, flask, requests, datasets
4	Operator	Download dataset once on each client machine: python -c "from datasets import load_dataset; load_dataset('imdb')"	IMDb dataset cached locally — no internet needed during demo

5	Operator	Find Machine A's local IP: ipconfig — note IPv4 address	IP noted — e.g. 192.168.1.42
6	Operator	Edit shared/config.py — set COORDINATOR_IP to Machine A's IP	All clients will now point to correct coordinator address

2.2 Pre-Training the Baseline Model

The baseline model is trained on the full 15,000 sample dataset for 3–5 epochs on Machine A. This checkpoint is committed to the repo and loaded by the coordinator at startup.

#	Actor	Action	System Response
1	Operator	Run on Machine A: python pretrain.py --epochs 5 --output checkpoint.pt	Training begins on full dataset using GPU
2	Coordinator (script)	Trains TransformerClassifier for 5 epochs on combined IMDb data	Model converges to ~65–70% accuracy baseline
3	Operator	Verify checkpoint: python verify_checkpoint.py --file checkpoint.pt	Prints: 'Checkpoint valid. Accuracy on val set: 67.3%'
4	Operator	Commit checkpoint to repo: git add checkpoint.pt && git commit -m 'Add pretrained baseline checkpoint'	checkpoint.pt available to all machines via git pull

Why pre-train at all

Starting from random weights means round 1 accuracy is ~50% (coin-flip). Starting from a 67% baseline means the federated rounds push it toward 75–80%, which is a visible and convincing improvement curve on camera. Same principle as YOLOv8 transfer learning — the system learns faster from a warm start.

3. Flow 1 — Coordinator Startup

The coordinator starts first. Clients cannot register until this flow completes.

#	Actor	Action	System Response
1	Operator	Navigate to coordinator/: cd coordinator	Terminal in coordinator directory
2	Operator	Start server: python server.py	Flask server initialises
3	Coordinator	Loads checkpoint.pt from disk into global model	TransformerClassifier loaded with pre-trained weights
4	Coordinator	Initialises SQLite database — creates credits table if not exists	database.db ready for credit logging
5	Coordinator	Sets round_number = 0, registered_clients = [], submitted_this_round = []	State initialised — server ready
6	Coordinator	Starts Flask on host='0.0.0.0', port=5000	Terminal prints: 'Net-Neutral AI Coordinator running on 0.0.0.0:5000'
7	Coordinator	Awaits incoming connections — all endpoints now active	Terminal prints: 'Waiting for clients to register...'

4. Flow 2 — Client Registration

All 3 clients register before any training begins. Registration must complete on all clients before the Operator triggers round 1.

#	Actor	Action	System Response
1	Operator	On each client machine: python client/client.py --id client_A (or B / C)	Client script starts, reads config.py for coordinator IP
2	Client A/B/C	POST /register with { client_id: 'client_A' }	Coordinator receives registration request

3	**Coordinator**	Adds client_id to registered_clients list	Coordinator terminal prints: 'client_A registered. (1/3 clients ready)'
4	**Coordinator**	Returns { status: 'ok', round: 0, message: 'Registered. Waiting for round start.' }	Client receives confirmation
5	**Client A/B/C**	Client terminal prints: 'Registered with coordinator. Waiting for round 1 to begin...'	Client enters polling loop — GET /status every 5 seconds
6	**Coordinator**	Once all 3 clients registered, terminal prints: 'All 3 clients ready. Type start_round to begin.'	System ready for round 1

Polling behaviour between rounds

After registering (and after each round submission), clients poll GET /status every 5 seconds. Their terminal prints 'Waiting for round [N]... (Xs elapsed)' updating each poll. This keeps the terminals active and visually interesting in the demo video without any manual input from the client operator.

5. Flow 3 — Training Round (Repeated 5 Times)

This is the core loop of the system. It repeats identically for rounds 1 through 5. The only difference between rounds is the starting weights — each round begins from the previous round's global model.

5.1 Round Start — Manual Trigger

#	Actor	Action	System Response
1	Operator	Types in coordinator terminal: start_round	Coordinator validates: all registered clients accounted for
2	Coordinator	Increments round_number (e.g. round_number = 1)	Internal state updated
3	Coordinator	Resets submitted_this_round = [], starts 90-second submission timer	Timeout clock begins
4	Coordinator	Sets round_status = 'active' in status response	Clients polling /status now receive round_status: 'active'
5	Coordinator	Terminal prints: 'Round 1 started. Waiting for client submissions (90s timeout)...'	Round is live

5.2 Client Detects Round Start

#	Actor	Action	System Response
1	Client A/B/C	Polls GET /status — receives round_status: 'active', round: 1	Client detects new round has begun
2	Client A/B/C	Downloads latest global model: GET /model	Coordinator streams checkpoint.pt binary to client
3	Client A/B/C	Loads received weights into local TransformerClassifier via torch.load	Local model now starts from global checkpoint

4	Client A/B/C	Terminal prints: 'Round 1 started. Downloading global model... Done.'	Client ready to train

5.3 Local Training

#	Actor	Action	System Response
1	Client A/B/C	Loads local data shard from DATA_PATH — 5000 samples	DataLoader initialised with batch_size=32
2	Client A/B/C	Records start_time = time.time()	Timer started
3	Client A/B/C	Runs 2 epochs of local training — Adam optimiser, CrossEntropyLoss	Gradients computed, weights updated locally
4	Client A/B/C	After each epoch, prints: 'Epoch 1/2 — Loss: 0.4821'	Training progress visible in terminal
5	Client A/B/C	Records time_seconds = time.time() - start_time, samples_trained = 5000	Metadata ready for submission

5.4 Weight Submission

#	Actor	Action	System Response
1	Client A/B/C	Saves model weights to temp file: torch.save(model.state_dict(), tmp.pt)	Binary weights file created locally
2	Client A/B/C	POST /submit — multipart: weights file + { client_id, round, samples_trained, time_seconds }	Request sent to coordinator
3	Coordinator	Receives weights file — saves to temp location	File stored, not yet averaged
4	Coordinator	Appends client_id to submitted_this_round list	Coordinator terminal prints: 'client_A submitted. (1/3 received)'

5	Coordinator	Calculates credits: points = floor(5000 / 5) = 1000	Credits computed
6	Coordinator	Writes to SQLite: INSERT into credits (client_id, round, samples, time, points)	Credit ledger updated
7	Coordinator	Returns { credits_earned: 1000, round: 1, status: 'received' } to client	Client receives acknowledgement
8	Client A/B/C	Terminal prints submission summary block (see TRD Section 5.3 for format)	Client enters polling loop waiting for round 2

5.5 FedAvg & Global Evaluation

Triggered automatically once all 3 clients submit, or when the 90-second timeout expires.

#	Actor	Action	System Response
1	Coordinator	Detects all 3 clients submitted (or timeout fires)	Proceeds with whoever submitted
2	Coordinator	Loads all received weight files into state_dicts	N state_dicts in memory (N = clients who submitted)
3	Coordinator	Runs FedAvg: for each layer key, global_weight = mean of all client weights	New global state_dict computed
4	Coordinator	Loads averaged weights into global model	Global model updated
5	Coordinator	Evaluates global model on 2000-sample validation set	Computes accuracy — e.g. 71.4%
6	Coordinator	Deletes temp weight files from disk	Disk cleaned up
7	Coordinator	Saves updated global model to checkpoint.pt	New checkpoint ready for next round

8	Coordinator	Terminal prints round summary	See format below

```
============================================
 Net-Neutral AI   |   COORDINATOR   |   Round 1 Complete
============================================
 Clients submitted : 3 / 3
 FedAvg            : COMPLETE
 Global accuracy   : 71.4%   (was 67.3% last round)
 Accuracy delta    : +4.1%
--------------------------------------------
 Leaderboard:
   1. client_C   —   1000 pts   (total: 1000)
   2. client_A   —   1000 pts   (total: 1000)
   3. client_B   —   1000 pts   (total: 1000)
============================================
 Type 'start_round' to begin Round 2...
```

9	Coordinator	Sets round_status = 'complete' — clients polling /status now see round complete	Clients exit polling loop and wait for next round_status: 'active'

6. Flow 4 — Rounds 2 Through 5

Rounds 2 through 5 are identical to Flow 3 with one difference: the global model starts from the previous round's checkpoint, not the original pre-trained baseline. This is what produces the visible accuracy climb.

Round	Starting Point	Expected Accuracy Range	Operator Action Required
Round 1	Pre-trained checkpoint (~67%)	69–72%	Type: start_round
Round 2	Round 1 global model	72–75%	Type: start_round
Round 3	Round 2 global model	74–77%	Type: start_round
Round 4	Round 3 global model	76–78%	Type: start_round
Round 5	Round 4 global model	77–80%	Type: start_round

Note on accuracy ranges

These are estimates based on IID data split and 2-epoch local training. Actual values may vary. What matters for the demo is an upward trend — even +1–2% per round is convincing. If accuracy plateaus early, do not worry — the system is still working correctly.

7. Flow 5 — Round 5 Completion & System Shutdown

#	Actor	Action	System Response
1	Coordinator	Round 5 FedAvg completes — global accuracy computed	Final accuracy logged
2	Coordinator	Queries SQLite for cumulative credits per client	Full leaderboard computed across all 5 rounds
3	Coordinator	Prints final summary to terminal	See format below

```
============================================
 Net-Neutral AI   |   FINAL RESULTS
============================================
 Total rounds completed : 5 / 5
```

```
 Final global accuracy  : 79.1%
 Starting accuracy      : 67.3%
 Total improvement      : +11.8%
-----------------------------------------
 FINAL LEADERBOARD:
   Rank 1  |  client_C  |  5100 pts
   Rank 2  |  client_A  |  5000 pts
   Rank 3  |  client_B  |  4900 pts
-----------------------------------------
 Accuracy per round:
   Round 1: 71.4%  |  Round 2: 73.8%
   Round 3: 75.9%  |  Round 4: 77.4%
   Round 5: 79.1%
=========================================
 All training complete. Shutting down in 10 seconds...
```

4	**Coordinator**	Sets round_status = 'done' — broadcasts shutdown signal via /status	Clients receive shutdown signal on next poll
5	**Client A/B/C**	Polls /status — receives round_status: 'done'	Client terminal prints: 'All rounds complete. Shutting down.'
6	**Client A/B/C**	Client script exits cleanly — sys.exit(0)	Client terminals show final state and close
7	**Coordinator**	After 10 second delay, Flask server shuts down — sys.exit(0)	Coordinator terminal shows final state and closes

Recording tip

Stop the screen recording 3–5 seconds after all terminals show their final state. Do not cut the recording the moment shutdown begins — let the final leaderboard sit on screen for a moment. That final frame is what judges screenshot.

8. Flow 6 — Error Handling & Edge Cases

8.1 Client Crashes or Times Out Mid-Round

#	Actor	Action	System Response
1	Client X	Crashes or freezes — never sends POST /submit	Coordinator's 90-second timer is running
2	Coordinator	90-second timer expires — submitted_this_round has fewer than 3 entries	Coordinator logs: 'Timeout — client_X did not submit this round'
3	Coordinator	Proceeds with FedAvg using only submitted clients — N = 2 instead of 3	FedAvg runs on partial client set
4	Coordinator	Prints: 'Warning: client_X timed out. FedAvg running with 2/3 clients.'	Visible in coordinator terminal
5	Coordinator	Awards zero credits to timed-out client for this round	SQLite log shows 0 points for that round
6	Coordinator	Round completes normally — timed-out client can rejoin next round by polling /status	System continues without interruption

8.2 Client Cannot Reach Coordinator

#	Actor	Action	System Response
1	Client A/B/C	POST /register fails with connection error	Client terminal prints: 'Cannot reach coordinator at [IP]:5000'
2	Client A/B/C	Client retries every 10 seconds for up to 3 attempts	Terminal prints: 'Retry 1/3...'
3	Client A/B/C	After 3 failed attempts, prints: 'Connection failed. Check config.py COORDINATOR_IP and ensure server is running.'	Client exits with helpful error message

Most likely cause

Either config.py has the wrong IP, the coordinator Flask server is not running yet, or Windows Firewall is blocking port 5000 on Machine A. Check in that order.

8.3 FedAvg Receives a Corrupted Weight File

#	Actor	Action	System Response
1	Coordinator	torch.load on a received file raises an exception	Coordinator catches the error in try/except
2	Coordinator	Logs: 'Warning: corrupted weights from client_X — excluding from this round's FedAvg'	Coordinator terminal shows warning
3	Coordinator	Proceeds with remaining valid weight files — N decremented by 1	FedAvg continues with clean files only

9. System State Machine

The coordinator is always in exactly one of these states. The client's behaviour depends on which state the coordinator is in.

State	Coordinator Behaviour	Client Behaviour	Transition
IDLE	Waiting for clients to register. Accepts /register only.	Not started yet.	All 3 clients register → READY
READY	All clients registered. Waiting for Operator to type start_round.	Polling /status every 5s. Printing 'Waiting for round N...'	Operator types start_round → ROUND_ACTIVE
ROUND_ACTIVE	90s timer running. Accepting /submit from clients.	Training locally. Will POST /submit when done.	All submit OR timeout → AGGREGATING
AGGREGATING	Running FedAvg + evaluation. Not accepting /submit.	Polling /status. Sees round_status: 'aggregating'.	FedAvg complete → READY (if rounds remain) or DONE
DONE	Printed final leaderboard. Broadcasting shutdown signal.	Received shutdown signal. Printing final state.	10s delay → all processes exit

10. Pre-Recording Checklist

Run through this list in order on the day of recording. Do not skip steps.

#	Check	Who	Pass Condition
1	All machines on same WiFi network	All	Machines can ping each other
2	config.py has correct COORDINATOR_IP on all machines	Operator	IP matches Machine A's ipconfig output
3	checkpoint.pt exists in coordinator/ on Machine A	Operator	File present, verify_checkpoint.py passes
4	All venvs activated on all machines	All	python --version shows correct environment
5	IMDb dataset cached on all client machines	ENTC members	load_dataset('imdb') runs without internet
6	Port 5000 open on Machine A firewall	ENTC1	Browser on Machine B hits http://[IP]:5000/status
7	Windows Terminal installed on Machine A for tiled view	ENTC1	4 panes visible on one screen
8	OBS or Xbox Game Bar ready to record	Operator	Test recording works before demo run
9	Do one full dry run — all 5 rounds	All	System completes without crash
10	Accuracy improves across rounds in dry run	CS (Janhvesh)	Round 5 accuracy > Round 1 accuracy
11	Record the actual demo run	Operator	Clean recording, no crashes, leaderboard visible at end

The dry run is not optional

Do not record the first run. Do a full dry run the day before, fix anything that breaks, then record the clean run. The video is your only deliverable — it has to be perfect.